

Documentacion Tecnica

App Museo AR

Resumen tecnico del proyecto desarrollado en la rama dev, con foco en librerias, versiones, funcion de cada dependencia, arquitectura, build, pruebas y recomendaciones.

Dato	Valor
Rama analizada	dev
Commit analizado	5053a80
Fecha de generacion	2026-08-03
Fuentes principales	package.json, pnpm-lock.yaml, README.md, ios/README.md, ios/project.yml, vite.config.ts, tsconfig*.json, .github/workflows/*.yml y codigo fuente en src/ e ios/.

1. Resumen ejecutivo

App Museo AR es una aplicación de realidad aumentada y recorrido panorámico 360 grados. La rama dev contiene una experiencia web para Android compatible con WebXR y ARCore, un fallback web para iPhone/iPad mediante AR Quick Look y una aplicación nativa iOS en la carpeta ios/ que replica las funciones principales con ARKit y RealityKit.

- La experiencia principal permite detectar una superficie horizontal, fijar una malla de 1 x 1 metro, colocar modelos 3D, ajustar escala de 1 cm a 1 m, rotar con gestos, aplicar corte vertical y activar efectos de partículas y sonido.
- El recorrido 360 incluye tres escenas de Paranal, puntos de interés, navegación entre paradas, zoom táctil, recentrado, pantalla completa y persistencia de progreso local.
- El proyecto mantiene pruebas unitarias para lógica de AR, estado, superficies, rotación, oclusión, catálogo de modelos, interacciones, progreso y panorama.

2. Stack y versiones

2.1 Dependencias npm directas

Librería	Version declarada	Version bloqueada	Tipo	Función en el proyecto
three	^0.180.0	0.180.0	Runtime	Motor 3D usado para escenas WebXR, renderizado WebGL, carga GLB con GLTFLoader, partículas, previsualización de modelos, panorama 360 y exportación/generación de activos.
qrcode	^1.5.4	1.5.4	Dev / scripts	Genera códigos QR para la aplicación y para la ruta dev durante el flujo de generación de activos.
vite	^7.1.7	7.3.6	Build / dev	Servidor de desarrollo, empaquetado de producción y gestión de base path para GitHub Pages y despliegue dev.
typescript	^5.9.2	5.9.3	Build / calidad	Tipado estricto de la aplicación web; no emite JS directamente y valida el código antes del bundle.
vitest	^3.2.4	3.2.7	Testing	Ejecuta las pruebas unitarias TypeScript en entorno Node para lógica de AR, panorama y utilidades.
@types/three	^0.180.0	0.180.0	Tipos	Tipos TypeScript de Three.js, incluyendo clases matemáticas, escena, cámaras, materiales y loaders usados por el código.
@types/webxr	^0.5.23	0.5.24	Tipos	Define tipos de WebXR, hit test, anchors y depth sensing utilizados por la experiencia AR.
@types/node	^24.5.2	24.13.3	Tipos	Tipos para scripts Node y configuración Vite/Vitest.
@types/qrcode	^1.5.5	1.5.6	Tipos	Tipos para el paquete qrcode usado en scripts de generación.

Nota: las versiones bloqueadas proceden de pnpm-lock.yaml. El campo de dependencias usa rangos con ^, por lo que una instalación futura podría resolver versiones menores o patch más recientes si se actualiza el lockfile.

2.2 Herramientas y plataforma web

Elemento	Version / condicion	Funcion
Node.js	22 segun README y CI	Runtime de desarrollo, scripts de generación y ejecución de Vite/Vitest.
pnpm	11.9.0 en CI; lockfile v9.0	Instalación reproducible con pnpm-lock.yaml. pnpm-workspace.yaml permite build de esbuild.
ES2022 / DOM	tsconfig.app.json	Target de la app web, APIs del navegador, DOM iterable y módulo ESNext con resolución Bundler.
ES2023	tsconfig.node.json	Target para configuración Node/Vite.
GitHub Pages	pages.yml	Prueba y compila main y dev; publica main en raíz y dev en /dev/.

2.3 Frameworks iOS y Apple

Framework / herramienta	Version / target	Funcion
Swift	SWIFT_VERSION 5.0	Lenguaje de la app nativa iOS.
iOS	Deployment target 17.0	Version minima de ejecucion declarada para app y tests.
Xcode	16 o posterior segun ios/README.md	Compilacion, firma, pruebas en simulador y despliegue en dispositivo.
XcodeGen	Requerido, no fijado en repo	Genera el proyecto Xcode a partir de ios/project.yml.
SwiftUI	SDK de Xcode/iOS	Interfaz de navegacion, pantallas AR y panorama.
ARKit	SDK de Xcode/iOS	World tracking, raycast sobre planos, scene reconstruction, scene depth, anchors y soporte AR nativo.
RealityKit	SDK de Xcode/iOS	Renderizado 3D nativo, entidades, materiales, ARView y escena panoramica.
Metal	SDK de Xcode/iOS	Shader custom para corte vertical del modelo mediante CustomMaterial.
Core Motion	SDK de Xcode/iOS	Orientacion del dispositivo para la vista panoramica 360.
UIKit / Combine / simd	SDK de Xcode/iOS	Integracion UIKit, estado reactivo y calculos vectoriales/matriciales.
XCTest	SDK de Xcode/iOS	Pruebas unitarias nativas para matematicas AR y estabilizacion de superficies.

3. APIs de navegador relevantes

API	Uso en la aplicacion	Riesgo / condicion
WebXR immersive-ar	Sesion AR web en Android compatible con ARCore.	Necesita HTTPS, navegador compatible y permisos de camara / spatial tracking.
WebXR Hit Test	Detecta superficies horizontales antes de colocar la malla.	Puede no estar disponible aunque WebXR exista.
WebXR Anchors	Mantiene la pose fisica del modelo y la malla si el dispositivo lo soporta.	El codigo conserva pose local cuando no hay anchor.
WebXR Depth Sensing	Solicita gpu-optimized para oclusion de objetos reales sobre modelo y malla.	Depende de Chrome, ARCore y hardware.
AR Quick Look	Fallback de iOS Safari para abrir USDZ.	No replica toda la interaccion WebXR; es un modo alternativo.
Web Audio	Sonidos sintetizados por modelo.	Puede requerir gesto de usuario y empieza desactivado.
Vibration API	Respuesta haptica breve en moviles compatibles.	No soportada en todos los navegadores.
Screen Wake Lock	Mantiene pantalla activa durante panorama.	Puede fallar por permisos o politicas del navegador.
Fullscreen API	Pantalla completa en la vista 360 cuando esta permitida.	No garantizada en todos los entornos moviles.
Device Orientation	Control por movimiento para panorama 360.	En iOS suele exigir permiso explicito del usuario.
localStorage	Persistencia de vista, progreso y escena panoramica.	Puede estar deshabilitado en navegacion privada o politicas restrictivas.

4. Arquitectura funcional

Area	Modulos / archivos	Responsabilidad
Entrada y UI web	src/main.ts, src/style.css, index.html	Orquesta vistas, dialogo de compatibilidad/camara, seleccion de modelo, controles de escala, corte, acciones, panorama y persistencia.
Experiencia AR WebXR	src/ar/xr-experience.ts	Gestiona sesion XR, renderer, hit test, anchors, malla, carga de modelos, oclusion, escala, corte, rotacion y ciclo de vida.
Compatibilidad y permisos	src/ar/platform.ts, src/ar/access-preflight.ts	Selecciona WebXR, Quick Look o modo no disponible; inspecciona permisos de camara y spatial tracking cuando el navegador lo permite.
Superficie y rotacion	src/ar/surface.ts, src/ar/rotation.ts	Valida horizontalidad, estabiliza muestras de hit test y calcula rotacion por gestos.
Catalogo e interaccion	src/ar/models.ts, src/ar/model-interaction.ts, src/ar/spores.ts, src/ar/model-sound.ts	Define cinco piezas, efectos, particulas, acciones, luz temporal, sonido y estados de bloqueo de animaciones.
Previsualizacion y fallback virtual	src/ar/model-preview.ts, src/virtual-experience.ts	Renderiza modelos sin AR y ofrece experiencia 3D alternativa cuando no se puede usar camara/AR.
Panorama 360	src/panorama-viewer.ts, src/panorama-tour.ts	Renderiza esfera panoramica, escenas, hotspots, navegacion, zoom, controles tactiles y movimiento del dispositivo.
Persistencia	src/progress-cache.ts	Normaliza y guarda progreso local de la app y ultima vista panoramica.
iOS nativo	ios/AppMuseoIOS/**	Implementa AR con ARKit/RealityKit, panorama con RealityKit/Core Motion, corte con Metal, UI SwiftUI y tests XCTest.
Scripts	scripts/*.mjs	Generacion de GLB, USDZ, QR, iconos, descargas panoramicas y validacion del proyecto iOS.

5. Activos y recursos

- Modelos web GLB: cosmic-flower.glb, crystal.glb, jellyfish.glb, mushroom.glb y totem.glb.
- Modelo iOS USDZ: mushroom.usdz, generado desde el GLB de la seta y normalizado a 20 cm segun ios/README.md.
- Panoramas: paranal-360.jpg, paranal-vlt-platform-360.jpg y paranal-residencia-360.jpg.
- PWA: manifest.webmanifest define nombre, idioma es, modo standalone, orientacion any, colores de tema y icono 1024 x 1024.
- Privacidad iOS: PrivacyInfo.xcprivacy declara que no se recopilan datos ni se realiza tracking; Info.plist declara uso de camara y requiere ARKit/arm64.

6. Build, pruebas y despliegue

Comando	Funcion
pnpm install	Instala dependencias desde pnpm-lock.yaml.
pnpm dev	Levanta Vite con host 0.0.0.0 para pruebas en red local/dispositivos.
pnpm build	Ejecuta tsc -b y vite build.
pnpm test	Ejecuta vitest run.
pnpm download:panoramas	Descarga/prepara escenas panoramicas adicionales desde fuentes ESO.
pnpm generate:assets	Genera activos web, QR, activos iOS y recursos compartidos.
pnpm generate:ios-assets	Genera USDZ e iconos iOS.
pnpm validate:ios	Valida consistencia del proyecto iOS y recursos esperados.

CI web: `.github/workflows/pages.yml` se activa en main, dev y manualmente. Descarga main y dev, usa pnpm 11.9.0 y Node 22, prueba y compila ambas ramas, genera assets y publica una web combinada con dev bajo `/AppMuseo_Andres/dev/`.

CI iOS: `.github/workflows/ios.yml` se activa en cambios de ios/, recursos clave y workflow. Corre en macos-15, muestra version de Xcode, selecciona un simulador iPhone disponible y ejecuta `xcodebuild test` con firma desactivada.

Cobertura observada: el repositorio contiene 77 declaraciones de pruebas entre suites TypeScript y XCTest. No se han ejecutado tests como parte de la generacion del PDF; esta documentacion refleja lectura estatica de la rama dev.

7. Riesgos tecnicos y recomendaciones

- Fijar packageManager en package.json, por ejemplo pnpm@11.9.0, para alinear maquinas locales y CI.
- Documentar una matriz minima de dispositivos/navegadores: Chrome Android con ARCore para WebXR, Safari iOS para Quick Look, iPhone/iPad iOS 17+ para app nativa, y casos con/sin LiDAR.
- Versionar o documentar la version recomendada de XcodeGen, ya que el proyecto iOS depende de ios/project.yml pero la herramienta no esta fijada.
- Separar en README o docs una tabla de capacidades por plataforma: WebXR Android, Quick Look iOS y app nativa iOS. Ayudara a explicar diferencias funcionales del fallback.
- Mantener pnpm-lock.yaml actualizado y revisar periodicamente Three.js, Vite y Vitest; son dependencias activas y conviene validar AR/render tras cualquier actualizacion.
- Incluir una guia breve de QA en dispositivo fisico con profundidad, permisos denegados, interrupciones de sesion, iluminacion baja, superficies brillantes y rotacion/escala/corte en extremos.
- Considerar snapshots visuales o pruebas E2E ligeras para UI critica del dialogo de camara, seleccion de modelo y panorama; hoy la mayor cobertura parece centrada en logica.
- Añadir un documento de licencia/atribucion de activos 3D y panoramas, manteniendo visibles las fuentes ESO y condiciones de uso.

8. Inventario rapido de dependencias por funcion

Funcion	Dependencias / APIs
Render 3D web	Three.js, WebGL via navegador, GLTFLoader.
AR web Android	WebXR immersive-ar, hit-test, anchors, depth-sensing, Three.js WebXR renderer.
Fallback iOS web	AR Quick Look con USDZ.
App iOS nativa	Swift, SwiftUI, ARKit, RealityKit, Metal, Core Motion, UIKit, Combine, simd.
Panorama 360	Three.js en web; RealityKit y Core Motion en iOS.
Sonido / haptica	Web Audio API, Vibration API.
Build web	Vite, TypeScript, Node 22, pnpm.

Funcion	Dependencias / APIs
Tests	Vitest para TypeScript, XCTest para iOS.
Generacion de activos	Three.js exporters, qrcode, Node.js scripts.
Publicacion	GitHub Actions, GitHub Pages, VITE_BASE_PATH.